

Week-7-S-2520080033

SESSION-13

Task-1: To parse Input Redirection, Open Files, Redirect stdin, Handle Missing Files, Restore Streams, Test File Inputs

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 256

int main() {
    char input[MAX_INPUT];
    char filename[MAX_INPUT];

    printf("Input Redirection Demonstration\n");
    printf("-----\n");

    printf("Enter a file for input redirection: ");
    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    /* Parse input redirection symbol '<' */
    char *redirect = strchr(input, '<');

    if (redirect == NULL) {
        printf("Error: Input redirection '<' not found.\n");
        return 1;
    }

    /* Extract filename after '<' */
    strcpy(filename, redirect + 1);

    /* Remove leading spaces */
    while (filename[0] == ' ') {
        memmove(filename, filename + 1, strlen(filename));
    }

    /* Remove trailing newline/spaces */
    filename[strcspn(filename, "\n")] = '\0';

    int len = strlen(filename);
    while (len > 0 && filename[len - 1] == ' ') {
        filename[len - 1] = '\0';
    }

    ^G Help      ^O Write Out  ^W Where Is   ^K Cut
    ^X Exit      ^P Read File  ^_ Replace    ^U Past
```

```

    if (dup2(fd, STDIN_FILENO) == -1) {
        perror("dup2");
        close(fd);
        close(saved_stdin);
        return 1;
    }

    close(fd);

    printf("\nContents read through redirected stdin:\n");
    printf("-----\n");

    /* Read data from redirected stdin */
    char buffer[128];
    ssize_t bytes_read;

    while ((bytes_read = read(STDIN_FILENO, buffer,
        sizeof(buffer) - 1)) > 0) {
        buffer[bytes_read] = '\0';
        printf("%s", buffer);
    }

    if (bytes_read == -1) {
        perror("read");
    }

    /* Restore original stdin */
    if (dup2(saved_stdin, STDIN_FILENO) == -1) {
        perror("Restoring stdin failed");
        close(saved_stdin);
        return 1;
    }

    close(saved_stdin);

    printf("\n-----\n");
    printf("stdin restored successfully.\n");
    printf("Input redirection completed successfully.\n");

    return 0;
}

```

```

root@Shamikahasini:~# nano skill7session13task1.c
root@Shamikahasini:~# gcc skill7session13task1.c -o skill7session13task1
root@Shamikahasini:~# ./skill7session13task1
=====
      Input Redirection Demonstration
=====
Enter a command with input redirection: cat < input.txt
Parsed input file: input.txt
Input redirection successful.
Reading contents through redirected stdin:
-----
Hello Ubuntu!
This is input redirection practical.
-----
Original stdin restored successfully.
Input redirection completed successfully.
root@Shamikahasini:~# |

```

Observation:

The program successfully parses the input redirection operator < and extracts the specified filename. The input file is opened using `open()` and its file descriptor is redirected to standard input using `dup2()`. The program then reads the file contents through `stdin`. If the specified file does not exist, an appropriate error message is

displayed using perror(). The original standard input is saved using dup() and restored after the redirection is completed.

Task-2; To Parse Output Redirection, Create Output Files, Redirect stdout, Handle Permissions, Verify Output Data, Test Redirection

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

#define MAX_INPUT 256

int main() {
    char input[MAX_INPUT];
    char filename[MAX_INPUT];

    printf("Output Redirection Demonstration\n");
    printf("-----\n");

    printf("Enter a command with output redirection: ");

    if (fgets(input, sizeof(input), stdin) == NULL) {
        printf("Error reading input.\n");
        return 1;
    }

    input[strcspn(input, "\n")] = '\0';

    /* Parse output redirection symbol '>' */
    char *redirect = strchr(input, '>');

    if (redirect == NULL) {
        printf("Error: Output redirection '>' not found.\n");
        return 1;
    }

    /* Extract filename after '>' */
    strcpy(filename, redirect + 1);

    /* Remove leading spaces */
    while (filename[0] == ' ') {
        memmove(filename, filename + 1, strlen(filename));
    }

    /* Remove trailing spaces */
    int len = strlen(filename);
```

GNU nano 7.2

```
if (strlen(filename) == 0) {
    printf("Error: No output filename specified.\n");
    return 1;
}

printf("Parsed output file: %s\n", filename);

/*
 * Open/create output file.
 * O_WRONLY -> write only
 * O_CREAT  -> create if file does not exist
 * O_TRUNC  -> clear existing contents
 */
int fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);

if (fd == -1) {
    perror("Error opening output file");
    printf("Output redirection failed.\n");
    return 1;
}

/* Save original stdout */
int saved_stdout = dup(STDOUT_FILENO);

if (saved_stdout == -1) {
    perror("dup");
    close(fd);
    return 1;
}

/* Redirect stdout to output file */
if (dup2(fd, STDOUT_FILENO) == -1) {
    perror("dup2");
    close(fd);
    close(saved_stdout);
    return 1;
}

close(fd);

/*
 * Everything printed to stdout from this point
 * Everything printed to stdout from this point
 * goes into the output file.
 */
printf("This data was redirected to the output file.\n");
printf("Output redirection is working successfully.\n");
printf("The stdout stream is currently connected to the file.\n");

/* Flush stdout before restoring it */
fflush(stdout);

/* Restore original stdout */
if (dup2(saved_stdout, STDOUT_FILENO) == -1) {
    perror("Restoring stdout failed");
    close(saved_stdout);
    return 1;
}

close(saved_stdout);

printf("stdout restored successfully.\n");
printf("Output redirection completed successfully.\n");

/* Verify output data by reopening the file */
FILE *verify = fopen(filename, "r");

if (verify == NULL) {
    perror("Verification failed");
    return 1;
}

printf("\nVerified output data from %s:\n", filename);
printf("-----\n");

char buffer[128];

while (fgets(buffer, sizeof(buffer), verify) != NULL) {
    printf("%s", buffer);
}

fclose(verify);

printf("-----\n");
```

```
root@Shamikahasini:~# nano skill7s13task2.c
root@Shamikahasini:~# gcc skill7s13task2.c -o skill7s13task2
root@Shamikahasini:~# ./skill7s13task2
=====
      Output Redirection Demonstration
=====
Enter a command with output redirection: echo Hello Ubuntu > output.txt
Parsed output file: output.txt
Stdout restored successfully.
Output redirection completed successfully.

Verified output data from output.txt:
-----
This data was redirected to the output file.
Output redirection is working successfully.
The stdout stream is currently connected to the file.
-----
Output file verified successfully.
```

Observation:

The program successfully parses the output redirection operator > and extracts the specified output filename. The output file is created or opened using `open()` with appropriate flags and permissions. Standard output is redirected to the file using `dup2()`, causing the program's output to be stored in the file. File-opening and permission errors are handled using `perror()`. After writing the data, the original stdout stream is restored and the output file is reopened to verify the redirected data.

SESSION-14

Task1: To Parse Append Operator, Open Files in Append Mode, Redirect Streams, Preserve Existing Data, Validate Appends, Test Scenarios

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

#define MAX_INPUT 256

int main() {
    char input[MAX_INPUT];
    char filename[MAX_INPUT];
    char data[MAX_INPUT];

    printf("Append Redirection Demonstration\n");
    printf("-----\n");

    /* Get append redirection input */
    printf("Enter command with append redirection: ");

    if (fgets(input, sizeof(input), stdin) == NULL) {
        printf("Error reading input.\n");
        return 1;
    }

    input[strcspn(input, "\n")] = '\0';

    /* Parse append operator >> */
    char *redirect = strstr(input, ">>");

    if (redirect == NULL) {
        printf("Error: Append operator '>>' not found.\n");
        return 1;
    }

    /* Extract filename after >> */
    strcpy(filename, redirect + 2);

    /* Remove leading spaces */
    while (filename[0] == ' ') {
        memmove(filename, filename + 1, strlen(filename));
    }
}
```

Help

Write Out

Where Is

Cut

^T

```
printf("This line was added without deleting existing data.\n");
printf("Append redirection using >> is working.\n");

fflush(stdout);

/* Restore stdout */
if (dup2(saved_stdout, STDOUT_FILENO) == -1) {
    perror("Restoring stdout failed");
    close(saved_stdout);
    return 1;
}

close(saved_stdout);

printf("stdout restored successfully.\n");
printf("Append operation completed.\n");

/* Verify the complete file */
FILE *verify = fopen(filename, "r");

if (verify == NULL) {
    perror("Verification failed");
    return 1;
}

printf("\nComplete contents of %s:\n", filename);
printf("-----\n");

char buffer[128];

while (fgets(buffer, sizeof(buffer), verify) != NULL) {
    printf("%s", buffer);
}

fclose(verify);

printf("-----\n");
printf("Existing data preserved and new data appended successfully.\n");

return 0;
}
```

```

root@Shamikahasini:~# nano skill7s14task1.c
root@Shamikahasini:~# gcc skill7s14task1.c -o skill7s14task1
root@Shamikahasini:~# echo "This is the original data." > append.txt
root@Shamikahasini:~# cat append.txt
This is the original data.
root@Shamikahasini:~# ./skill7s14task1
=====
Append Redirection Demo
=====
Enter command with append redirection: echo New data >> append.txt
Parsed output file: append.txt
Stdout restored successfully.
Append operation completed.

Complete contents of append.txt:
-----
This is the original data.
This line was added without deleting existing data.
Append redirection using >> is working.
-----
Existing data preserved and new data appended successfully.

```

Observation:

The program successfully parses the append redirection operator >> and extracts the output filename. The file is opened using O_WRONLY, O_CREAT, and O_APPEND, ensuring that new data is written at the end of the file without deleting existing contents. Standard output is redirected to the file using dup2(). After the append operation, the original stdout stream is restored and the complete file contents are read and displayed to verify that the existing data was preserved and the new data was appended successfully.

Task2: TO parse stderr Redirection, Redirect Error Streams, Capture Error Output, Verify File Contents, Handle Failures, Test Error Cases

```
GNU nano 7.2 skill7
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

#define MAX_INPUT 256

int main() {
    char input[MAX_INPUT];
    char filename[MAX_INPUT];

    printf("Standard Error Redirection Demonstration\n");
    printf("-----\n");

    printf("Enter command with stderr redirection: ");

    if (fgets(input, sizeof(input), stdin) == NULL) {
        printf("Error reading input.\n");
        return 1;
    }

    input[strcspn(input, "\n")] = '\0';

    /* Parse stderr redirection operator 2> */
    char *redirect = strstr(input, "2>");

    if (redirect == NULL) {
        printf("Error: stderr redirection operator '2>' not found.\n");
        return 1;
    }

    /* Extract filename after 2> */
    strcpy(filename, redirect + 2);

    /* Remove leading spaces */
    while (filename[0] == ' ') {
        memmove(filename, filename + 1, strlen(filename));
    }

    /* Remove trailing spaces */
    int len = strlen(filename);
```

```

while (len > 0 && filename[len - 1] == ' ') {
    filename[len - 1] = '\0';
    len--;
}

if (strlen(filename) == 0) {
    printf("Error: No error-output filename specified.\n");
    return 1;
}

printf("Parsed error output file: %s\n", filename);

/*
 * Open/create error output file.
 *
 * O_WRONLY -> write only
 * O_CREAT  -> create if it does not exist
 * O_TRUNC  -> overwrite previous contents
 */
int fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);

if (fd == -1) {
    perror("Error opening error-output file");
    return 1;
}

/* Save original stderr */
int saved_stderr = dup(STDERR_FILENO);

if (saved_stderr == -1) {
    perror("dup");
    close(fd);
    return 1;
}

/* Redirect stderr to the file */
if (dup2(fd, STDERR_FILENO) == -1) {
    perror("dup2");
    close(fd);
    close(saved_stderr);
    return 1;
}

```

⌘ G Help
⌘ O Write Out
⌘ W Where Is
⌘ K Cut
⌘ T Exe
⌘ X Exit
⌘ R Read File
⌘ \ Replace
⌘ U Paste
⌘ J Jus

```

fprintf(stderr, "Error: This message was redirected from stderr.\n");
fprintf(stderr, "Error: Demonstration of error stream capture.\n");

fflush(stderr);

/* Restore original stderr */
if (dup2(saved_stderr, STDERR_FILENO) == -1) {
    perror("Restoring stderr failed");
    close(saved_stderr);
    return 1;
}

close(saved_stderr);

printf("stderr restored successfully.\n");
printf("Error output redirection completed.\n");

/* Verify file contents */
FILE *verify = fopen(filename, "r");

if (verify == NULL) {
    perror("Verification failed");
    return 1;
}

printf("\nCaptured error output from %s:\n", filename);
printf("-----\n");

char buffer[128];

while (fgets(buffer, sizeof(buffer), verify) != NULL) {
    printf("%s", buffer);
}

fclose(verify);

printf("-----\n");
printf("Error output file verified successfully.\n");

return 0;
}

```

```

root@Shamikahasini:~# nano skill7s14task2.c
root@Shamikahasini:~# gcc skill7s14task2.c -o skill7s14task2
root@Shamikahasini:~# ./skill7s14task2
=====
Standard Error Redirection
=====
Enter command with stderr redirection: cat missing.txt 2> errors.txt
Parsed error output file: errors.txt
stderr restored successfully.
Error output redirection completed.

Captured error output from errors.txt:
-----
Error: Unable to open requested file.
Error: This message was redirected from stderr.
Error: Demonstration of error stream capture.
-----
Error output file verified successfully.
root@Shamikahasini:~# cat errors.txt
Error: Unable to open requested file.
Error: This message was redirected from stderr.
Error: Demonstration of error stream capture.
root@Shamikahasini:~# |

```

Observation:

The program successfully parses the stderr redirection operator 2> and extracts the specified error-output filename. The file is opened using open() and standard error is redirected to the file using dup2(). Error messages generated through stderr are captured in the specified file instead of being displayed directly on the terminal. The original stderr stream is restored after redirection, and the file is reopened to verify the captured error contents. Invalid redirection syntax and missing filenames are also handled appropriately.